Increment ++ and Decrement -- Operator in C++

In C++, we use the **increment** ++ and **decrement** -- operators to **increase** and **decrease** the value of a variable **by one** respectively.

```
int x = 10;
x = x+1;
or
x++;   <- post incr. op
or
++x;   <- pre incr. op
cout << x; // 11
```

```
int z = 90;
z = z-1;
or
z--;  // post decr. op
or
--z;  // pre decr op
cout << z; // 89
```

```
x = 10;
cout << x++; // 10
cout << x; // 11
```

```
int y = 20;
cout << ++y; // 21
cout << y; // 21
```

```
int a = 50;
int b = a++;
cout << a; // 51
cout << b; // 50
```

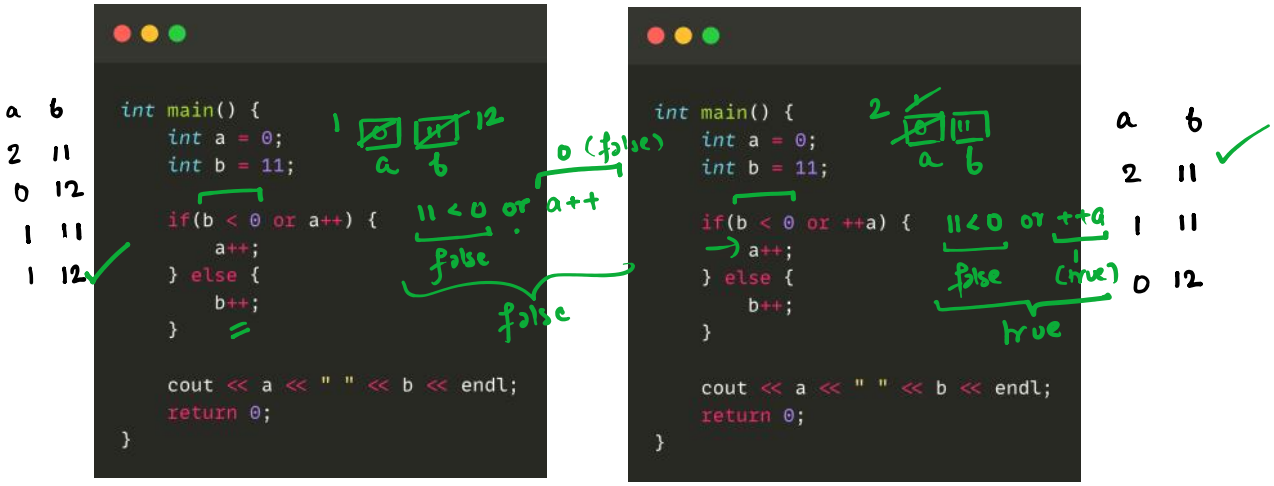
```
int c = 100;
int d = ++c;
cout << c; // 101
cout << d; // 101
```

```
int main() {
    int a = 0;
    int b = 10;

    if(a > 0 and a++) {
        a++;
    } else {
        b++;
    }

    cout << a << " " << b << endl;
    return 0;
}
```

a b
0 11
0 10

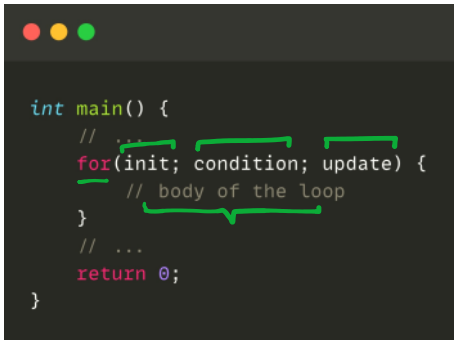
```
int main() {
    int a = 0;
    int b = 10;

    if(b > 0 or a++) {
        a++;
    } else {
        b++;
    }

    cout << a << " " << b << endl;
    return 0;
}
```

a b
1 10
2 10

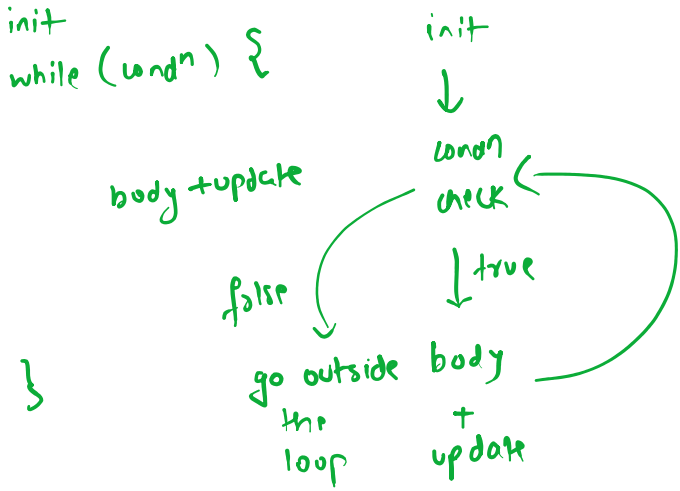

a b
2 11
0 12
1 11
1 12
int main() {
int a = 0;
int b = 11;
if(b < 0 or a++) {
a++;
} else {
b++;
}
cout << a << " " << b << endl;
return 0;
}
1
12
a
b
0 (false)
11 < 0 or a++
false
false
int main() {
int a = 0;
int b = 11;
if(b < 0 or ++a) {
a++;
} else {
b++;
}
cout << a << " " << b << endl;
return 0;
}
2
a
b
11 < 0 or ++a
false
(true)
true
a b
2 11
1 11
0 12

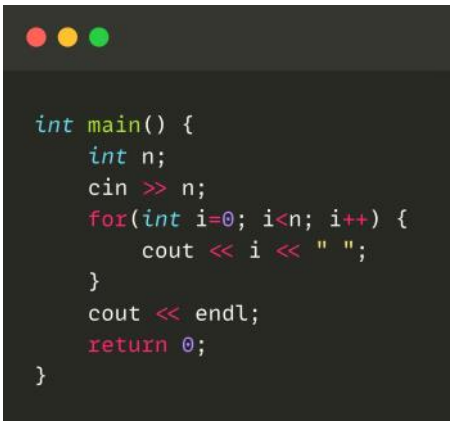
Other Loops in C++

➢ The for Loop

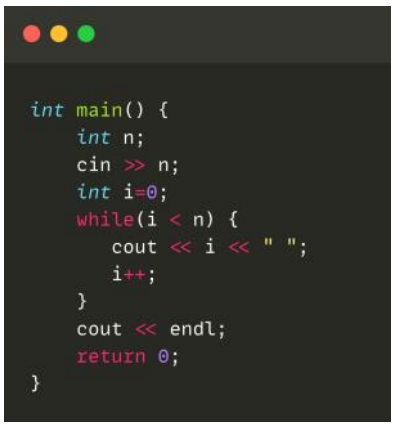


```
int main() {
    // ...
    for(init; condition; update) {
        // body of the loop
    }
    // ...
    return 0;
}
```

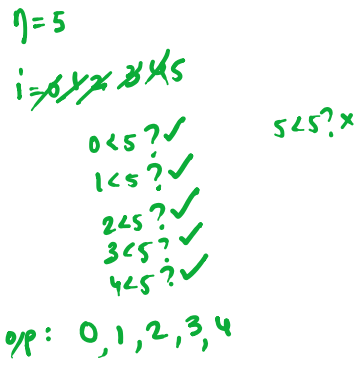




```
int main() {
    int n;
    cin >> n;
    for(int i=0; i<n; i++) {
        cout << i << " ";
    }
    cout << endl;
    return 0;
}
```



```
int main() {
    int n;
    cin >> n;
    int i=0;
    while(i < n) {
        cout << i << " ";
        i++;
    }
    cout << endl;
    return 0;
}
```



while vs for Loop

- Use a **while** loop if the loop condition or the update rule is **complex**.
- Use a **for** loop to iterate over a **sequence** of items.

➢ **[HW][R]The do-while Loop**

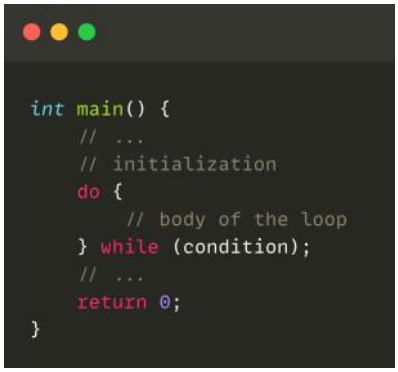
```
int main() {
    // ...
    // initialization
    do {
        // body of the loop
    } while (condition);
    // ...
    return 0;
}
```

```
int main() {
    int n;
    cin >> n;
    int i = 0;
    do {
      cout << i << " ";
      i++;
    } while(i < n);
    cout << endl;
    return 0;
}
```

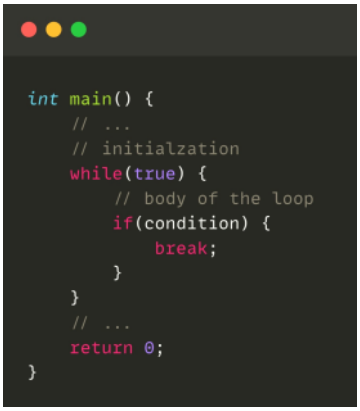
Unlike a **while** or a **for** loop, the body of the **do-while** loop will run **at-least once** irrespective of the loop condition.

```
int main() {
    int i = 5;
    do {
      cout << i << " ";
      i++;
    } while(i < 5);
    cout << endl;
    return 0;
}
```

We can **simulate** a do-while loop using a while loop.



```c
int main() {
    // ...
    // initialization
    do {
        // body of the loop
    } while (condition);
    // ...
    return 0;
}
```

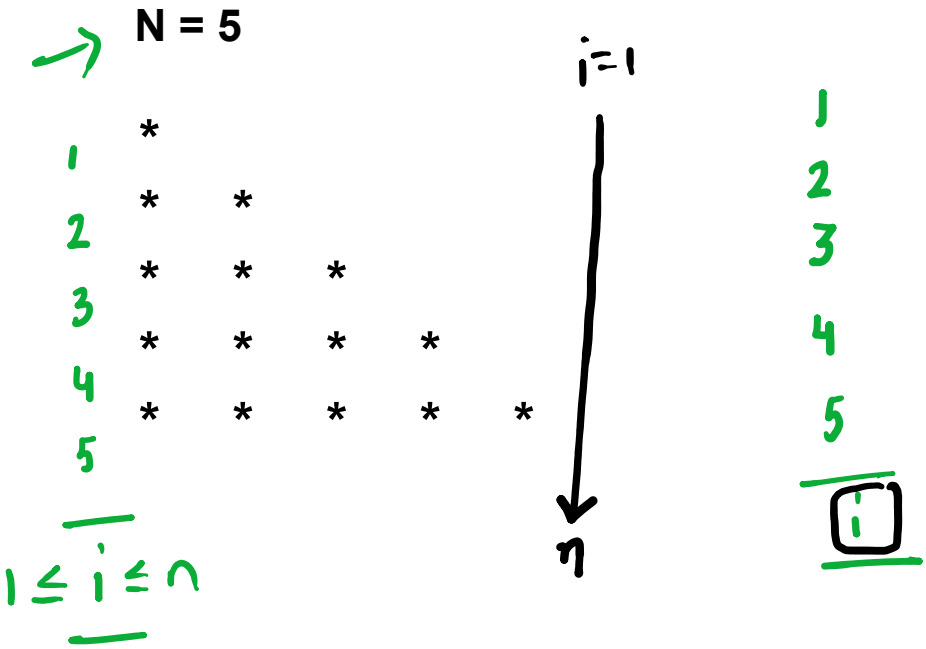


```c
int main() {
    // ...
    // initialzation
    while(true) {
        // body of the loop
        if(condition) {
            break;
        }
    }
    // ...
    return 0;
}
```

Pattern - 1

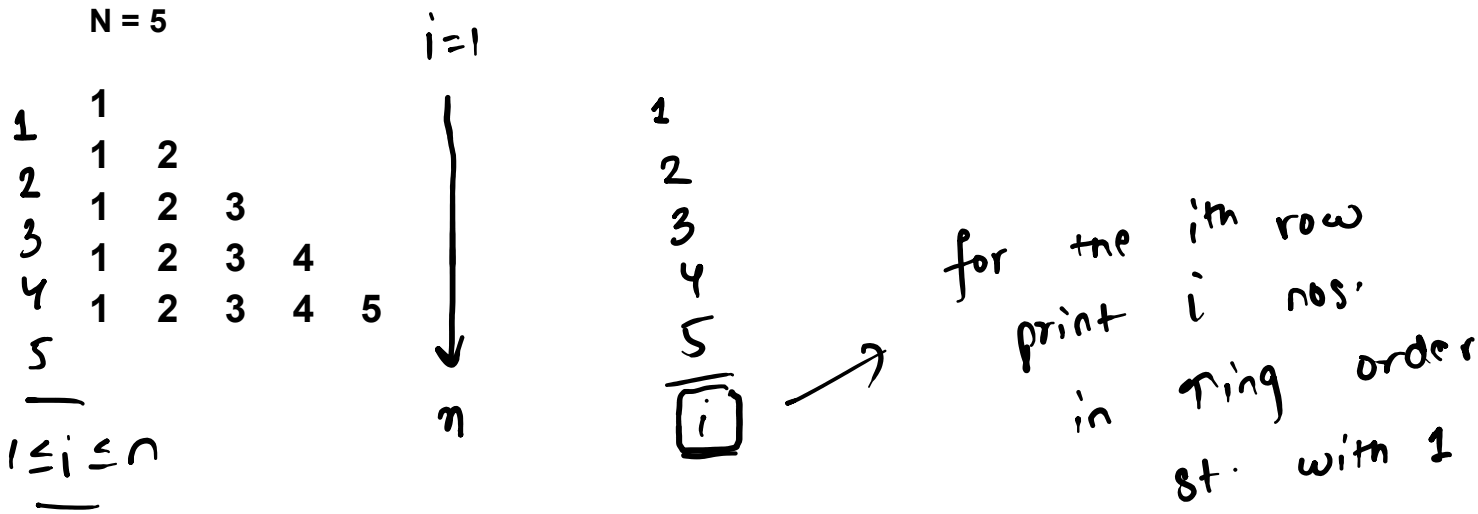
Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example



Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example



Pattern - 3

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

N = 4

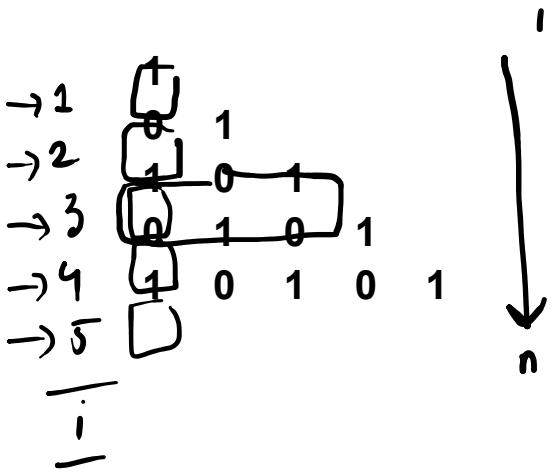
1
2 3
4 5 6
7 8 9 10

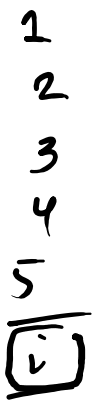
Pattern - 4

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

N = 5





even row ↓ 0

odd row ↓ 1

num = !num

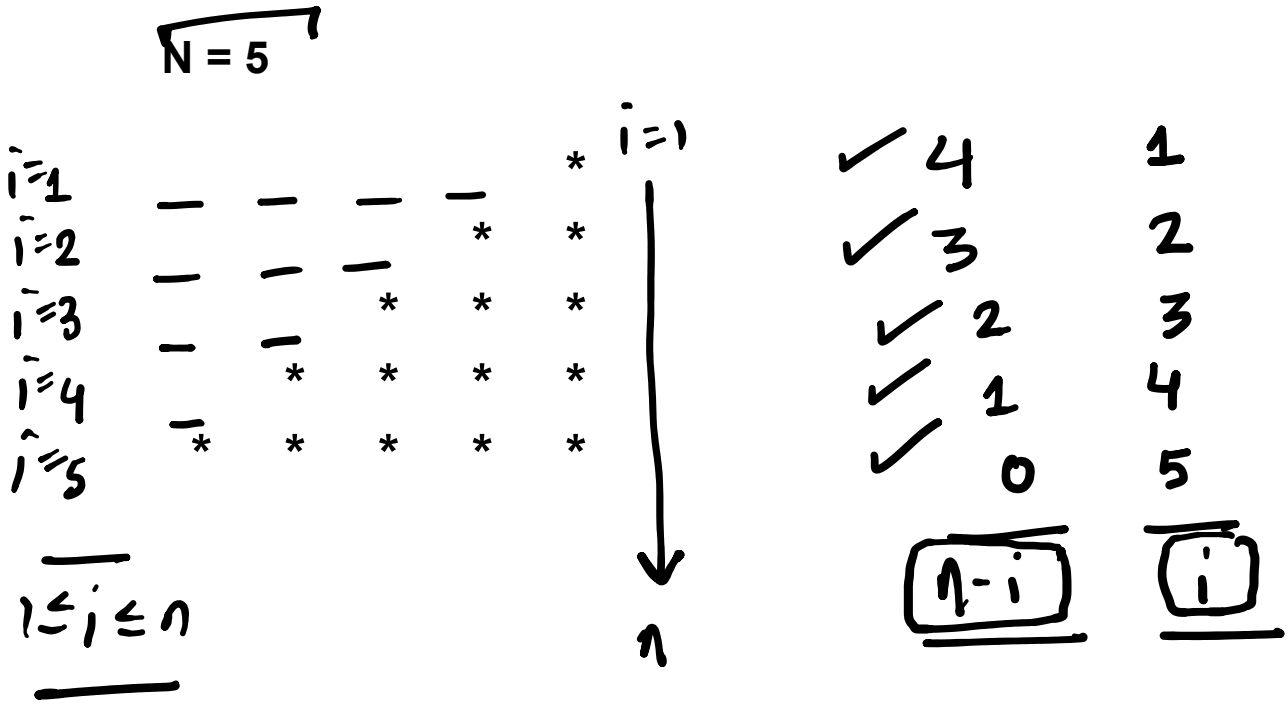
or

num = 1 - num;

Pattern - 5

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

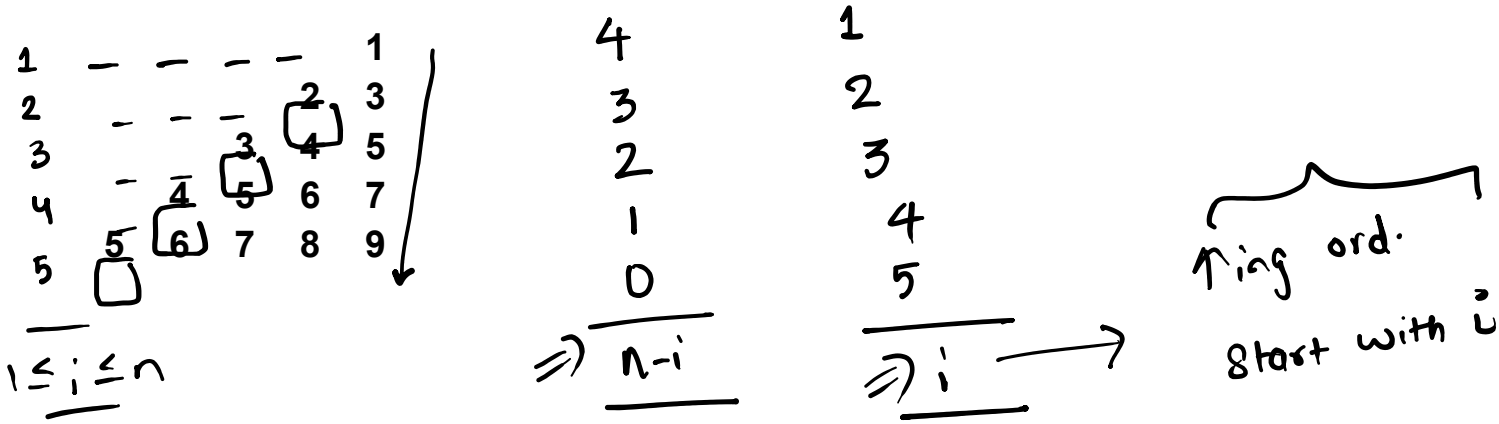


Pattern - 6

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

N = 5

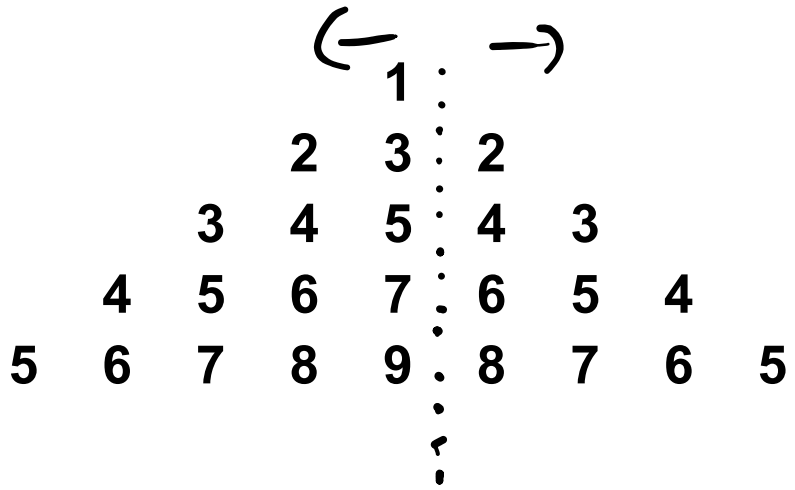


[R][HW] Pattern - 7

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

N = 5



Characters in C++

```
char ch;
cin >> ch;
cout << ch;
```

```
char ch;
ch = 'A';
cout << ch; // A
```

A ch

1B

```
char ch = 'B';
cout << ch; // B
```

ch B

1B = 8 - bits

n-bits ⇒ 2^n nos

00...0 ⇒ 0

11....1 ⇒ $2^n - 1$

ASCII

A → 65
B → 66
⋮
Z → 90

a → 97
b → 98
c → 99
⋮
z → 122

2 2 2 2 2 2 2 2 ⇒ 2^8 nos.

00000000 → 0
⋮
11111111 → 255

```
char ch = 'd';
```

ascii value of 'd' = 100

64 + 32 + 4
↓ ↓ ↓
2^6 2^5 2^2

0 1 1 0 0 1 0 0
7 6 5 4 3 2 1 0

8 - bits

'0' → 48
'1' → 49
'2' → 50
'3' ⋮
⋮ ⋮
'9' → 57

ch = '8' → 8

char int

'8' - '0'

56 - 48 = 8

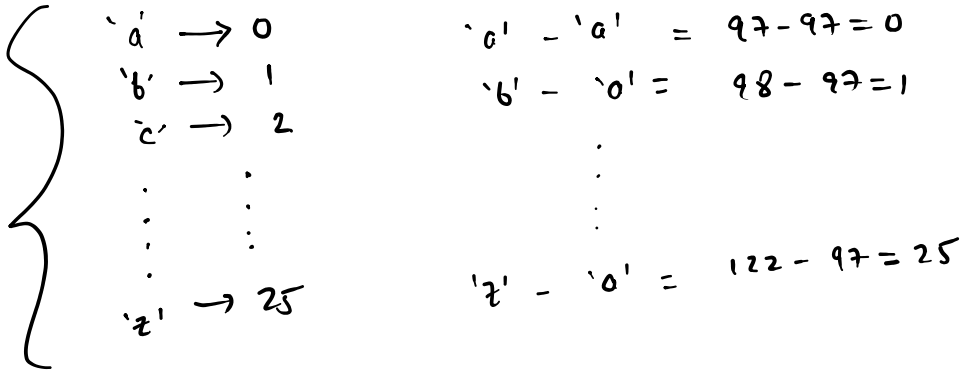
ch - '0' = ☐

(digit in char form) digit in int form

'6' → 6

'6' - '0'

54 - 48 = 6



i/p abcd$ o/p : 4

i/p ab_ cde$ o/p: 6

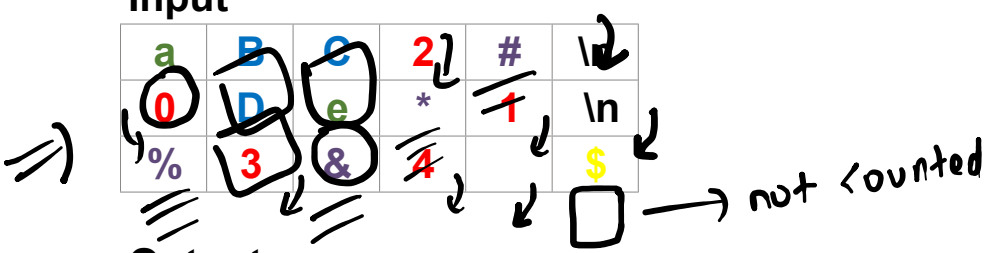
Count Types of Characters

Given a sequence of characters, design an algorithm to **count** the number of lowercase letters, uppercase letters, digits, whitespaces and special symbols.

note : assume the character sequence ends with '$'

Example

Input



Output

lowercase	uppercase	digits	whitespaces	special
2	3	5	3	4

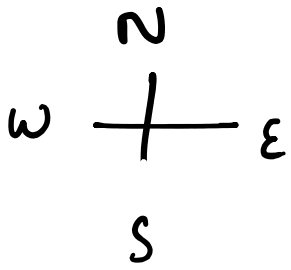
Minimize Path

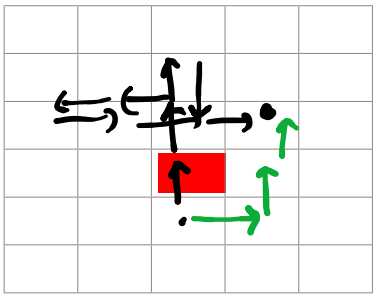
Input

N	N	N	S	W	W	E	E	E

Output

E	N	N





lexicographically smallest

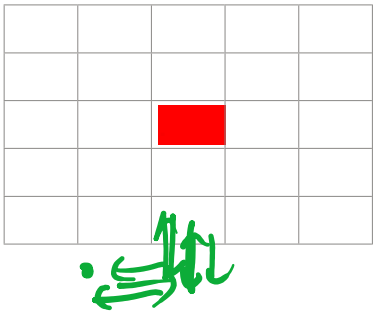
ENN
or
NNE
or
NEN

Input

S	N	S	N	S	W	E	W

Output

S	W



SW ✓
or
WS

Constants in C++